

軟體測試

教師：王凡

授課時間：每週二 9：10—12：00

博理館 212

課程代號：EE 5170

識別碼：921 U9550

民國壹百零貳年春季班

期末考

姓名：

科系年級：

學號：

考試中，請勿參閱資料。

1. Use the following predicate to answer the following questions.

$$p = (a \wedge b) \vee (b \wedge c) \vee (\neg a \wedge \neg c)$$

(a) Identify the clauses that determine predicate p. (2pts/2)

(b) Compute (and simplify) the conditions under which each of the clauses determines predicate p. (3pts/5)

(c) Write the complete truth table for all clauses. Label your rows starting from 1. Use the format in the example underneath the definition of combinatorial coverage in Section 3.2. That is, row 1 should be all clauses true. You should include columns for the conditions under which each clause determines the predicate, and also a column for the predicate itself. (4pts/9)

(d) Identify all pairs of rows from your table that satisfy general active clause coverage (GACC) with respect to each clause. (2pts/11)

(e) Identify all pairs of rows from your table that satisfy correlated active clause coverage (CACC) with respect to each clause. (2pts/13)

(f) Identify all pairs of rows from your table that satisfy restricted active clause coverage (RACC) with respect to each clause. (3pts/16)

(g) Identify all 3-tuples of rows from your table that satisfy general inactive clause coverage (GICC) with respect to each clause. Identify any infeasible GICC test requirements. (2pts/18)

(h) Identify all 3-tuples of rows from your table that satisfy restricted inactive clause coverage (RICC) with respect to each clause. Identify any infeasible RICC test requirements. (2pts/20)

2. Answer the following questions for the method twoPred() below:

```
public String twoPred (int x, int y) {  
    boolean z;  
    if (x < y)  
        if (x<3*y)  
            return "B";  
        else  
            z = true;  
    else  
        z = false;  
    if (z && x+y == 10)  
        return "A";  
    else  
        return "B";  
}
```

- (a) Identify test inputs for twoPred() that achieve Restricted Active Clause Coverage (RACC). (5pts/25)

- (b) Identify test inputs for `twoPred()` that achieve Restricted Inactive Clause Coverage (RICC). (5pts/30)

3. Consider the following deterministic finite state machine:

Current State	Condition	Next State
Idle	$a \wedge b$	Active
Idle	$\neg a \wedge \neg b$	WindDown
Active	$a \wedge \neg b$	Idle
Active	b	WindDown
WindDown	b	Idle

(a) Draw the finite state machine. (3pts/33)

- (b) This machine does not specify which conditions cause a state to transition back to itself. However, these conditions can be derived from the existing conditions. Derive the conditions under which each state will transition back to itself. (3pts/36)

(c) Find CACC tests for each transition from the Active state. (4points/40)

4. Use the following function to answer the following questions.

$$f = (\neg a \wedge \neg c \wedge \neg d) \vee (b \wedge c) \vee (b \wedge \neg c \wedge d)$$

(a) Draw the Karnaugh maps for f and $\neg f$. (2pts/42)

(b) Find the nonredundant prime implicant representation for f and $\neg f$. (2pts/44)

(c) Give a test set that satisfies implicant coverage (IC) for f . (2pts/46)

(d) Give a test set that satisfies unique true point coverage (UTPC) for f . (2pts/48)

- (e) Give a test set that satisfies Corresponding unique true point and near false point pair coverage (CUTPNFP) for f . (2pts/50)

5. Derive input space partitioning tests for the GenericStack class with the following method signatures:

- public GenericQueue ();
- public void Enqueue (Object X);
- public Object Dequeue ();
- public boolean IsEmpty ();

Assume the usual semantics for the queue. Try to keep your partitioning simple, choose a small number of partitions and blocks.

(a) Define characteristics of inputs. (5pts/55)

(b) Partition the characteristics into blocks (5pts/60)

(c) Define values for the blocks (5pts/65)

6. Enumerate all 16 tests to satisfy the pair-wise (PWC) criterion for the second categorization of TriTyp's inputs in the following Table.

Partition	b_1	b_2	b_3	b_4
q_1 = "Length of Side 1"	greater than 2	equal to 2	equal to 1	less than 1
q_2 = "Length of Side 2"	greater than 2	equal to 2	equal to 1	less than 1
q_3 = "Length of Side 3"	greater than 2	equal to 2	equal to 1	less than 1

Use the values in the following Table. (5pts/70)

Parameter	b_1	b_2	b_3	b_4
Side 1	3	2	1	-1
Side 2	4	2	1	-1
Side 3	3	2	1	0

7. Explain the concept of “killing a mutant” in mutation testing ? (4pts/74)

8. Explain the following terms:

(a) Mutation Coverage: (2pts/76)

(b) Mutation Operator Coverage: (2pts/78)

(c) Mutation Production Coverage: (2pts/80)

9. Provide reachability conditions to kill mutants 1, 2, 3, and 4 in the following method.
(5pts/85)

Original method	with Embedded Mutants
<pre>int Min(int A, int B) { int minVal; minVal = A; if (B<A) { minVal = B; } Return (minVal); } // endMin()</pre>	<pre>int Min (int A, int B) { int minVal; minVal = A; Δ1 minVal = B; if (B<A) { Δ2 if (B>=A) { minVal = B; Δ3 Bomb(); Δ4 minVal = 0; } return (minVal); } // Min()</pre>

10. Following question 9, please provide infection conditions to kill mutants 1, 2, 3, and 4.
(5pts/90)

11. Following question 9, please provide propagation conditions to kill mutants 1, 2, 3, and 4. (5pts/95)

12. Following question 9, please provide test case values to kill mutants 1, 2, 3, and 4. (5pts/100)